

SLM-to-LLM: Language Modeling, Practice, and Modern LLM Systems

From Shannon and n-grams to Transformers, RAG, reasoning, agents, and Myanmar-language practice

Dr. Ye Kyaw Thu

Lab Leader, Language Understanding Lab, AI Mario, Myanmar

Visiting Professor, Core-AI Research Team, NECTEC, Thailand

[Homepage](#) [GitHub](#)

1. What exactly is a language model?

Probability, prediction units, and objectives

Language modeling in one sentence

- A language model assigns a probability to a sequence and therefore a next-token distribution.
- It turns linguistic context into a numerical preference over possible continuations.
- Training = learn parameters that make observed text probable; inference = use the learned distribution.

Sequence probability

$$P(w_{1:T}) = \prod_{t=1}^T P(w_t \mid w_{<t})$$

Symbols

$w_{1:T}$: token sequence; w_t : token at position t ; $w_{<t}$: all earlier tokens; T : sequence length.

- The chain rule turns a whole-sequence probability into repeated next-token probabilities.
- LM families mainly differ in how they represent and compute the conditional distribution.

Cross-entropy loss

$$\mathcal{L} = -\frac{1}{N} \sum_{t=1}^N \log P_{\theta}(w_t \mid w_{<t})$$

Symbols

$\mathcal{L}$: mean negative log-likelihood; N : number of prediction events; w_t : target token; $w_{<t}$: context before the target.

- Lower loss means more probability mass is assigned to the observed targets.
- Perplexity is the exponential of the average negative log-likelihood.

Perplexity

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_t \log P(w_t \mid w_{<t}) \right)$$

Symbols

N : prediction events; w_t : target token; $w_{<t}$: preceding context; PPL: perplexity.

- PPL is an exponential transform of average negative log-likelihood.
- Compare PPL only under comparable prediction units and evaluation protocols.

When perplexity misleads

- A word-level LM and a subword LM may count different prediction events.
- A model can score a lower PPL by memorizing a narrow test distribution.
- For practical systems, task accuracy, calibration, latency, cost, and grounding also matter.

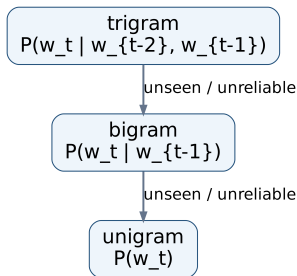
2. Statistical language models

Probability, sparsity, smoothing, and evaluation

The n-gram assumption

- Approximate history with the previous $n - 1$ tokens.
- Trigram example: $P(w_t \mid w_{t-2}, w_{t-1})$.
- This makes counting feasible, but throws away longer context.

Backoff is the core engineering idea



If a higher-order event is unseen or unreliable, back off to a lower-order distribution with a correction factor.

Why raw counts fail

- The space of possible n-grams grows exponentially with order.
- Many legitimate sequences never occur in finite corpora.
- Zero probability is catastrophic because one zero term makes the whole sentence probability zero.

Smoothing family

- Additive methods: simple, pedagogical, usually weak for real LMs.
- Good-Turing / Katz backoff: estimate unseen mass from count-of-counts.
- Kneser-Ney: re-estimate lower-order probabilities using continuation behavior.

Why Kneser-Ney mattered

- A word's usefulness as a continuation is not the same as its raw frequency.
- The lower-order distribution rewards diverse contexts rather than repeated appearances in a few contexts.
- Interpolated Kneser-Ney became a canonical strong statistical LM recipe.

Reference – Kneser & Ney (1995)

KenLM: build a 3-gram

Notebook / script pattern

```
lmplz -o 3 -S 16G -T tmp_kenlm --discount_fallback < train.txt > model.arpa
```

Practical note: the tutorial uses discount fallback for small-data robustness.

ARPA: model representation

Notebook / script pattern

```
\data\ngram 1=... ngram 2=... ngram 3=... \1-grams: \2-grams: \3-grams:
```

ARPA exposes: n-gram order, log probabilities, and backoff weights.

SRILM: read the score one word at a time

- Use the `-debug 2` option to expose the conditional probabilities used for each word.
- The debug trace exposes OOV handling and backoff events.
- Connect the trace directly to the chain-rule equation.

3. Neural language models

Distributed representations for language modeling

Bengio's neural LM

- Replace sparse discrete context counts with learned dense representations.
- A word's embedding can participate in many contexts.
- Generalization comes from parameter sharing rather than exact n-gram matches.

Reference – Bengio et al., 2003

Embedding lookup

- An integer token id selects a dense vector.
- Similar contexts can push related token representations together.
- Vocabulary size V and embedding dimension d define a $V \times d$ matrix.

Neural LM output

$$h_t = f_{\theta}(e(w_{t-k:t-1})); \quad P(w_t) = \text{softmax}(Wh_t + b)$$

Symbols

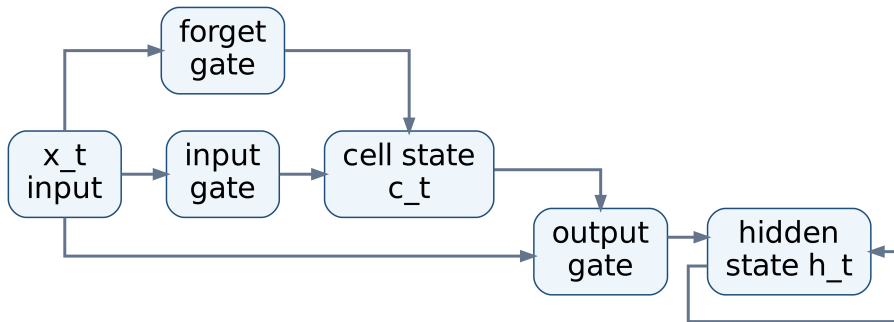
$e(\cdot)$: embedding lookup; $w_{t-k:t-1}$: previous k tokens; h_t : hidden state; f_{θ} : neural network with parameters θ ; W : output weight matrix; b : output bias vector.

- The hidden state is a learned summary of the context.
- Softmax converts output logits into a probability distribution over the vocabulary.

RNN recurrence

- Maintain a state h_t that carries information forward.
- Reuse the same parameters at every time step.
- This gives variable-length context, at least in principle.

LSTM: controlled information flow



The cell state and gates let the network decide what to keep, forget, and expose.

LSTM gates

- Forget gate: what old information should disappear?
- Input gate: what new information should be written?
- Output gate: what state should be exposed?

Training loop

Notebook / script pattern

```
logits, _ = model(x) loss = CrossEntropy(logits.view(-1,V), y.view(-1)) loss.backward();  
optimizer.step()
```

Objective: next-token likelihood, now parameterized by a neural network.

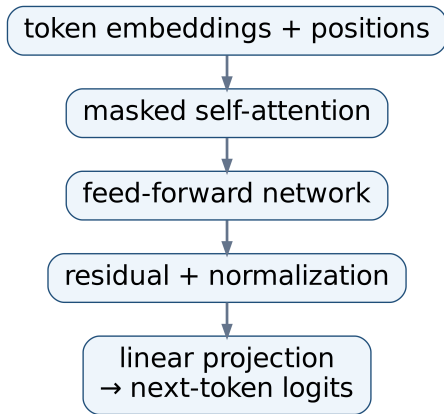
4. Transformers and the modern LM

The key idea: replace recurrence with content-addressable attention

Why attention changed the game

- Each token can directly interact with other tokens in the context.
- Training parallelizes across sequence positions.
- The same building block scales cleanly with model and data size.

Transformer decoder LM: the causal path



A causal LM stacks attention + feed-forward blocks; the final logits predict the next token.

Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V$$

Symbols

Q : queries; K : keys; V : values; d_k : key-vector dimension; M : attention mask; QK^T : pairwise compatibility scores.

- Scaling by $\sqrt{d_k}$ stabilizes the softmax inputs.
- In a causal LM, M prevents attending to future tokens.

Multi-head attention

- Run several attention projections in parallel.
- Different heads can specialize in different relations.
- Concatenate and project the head outputs.

Causal masking

- Token t may attend to positions $\leq t$.
- It must not see future labels during training.
- This makes teacher-forced next-token training consistent with generation.

RoPE (Rotary Position Embedding): the intuition

- Rotate query/key subspaces by position-dependent angles.
- Relative offsets emerge naturally from phase differences.
- Widely used in open decoder-only model families.

5. Tokenization is part of model architecture

Especially important for Myanmar and other lower-resource languages

Why tokens are a modeling decision

- The tokenizer determines sequence length.
- It determines the prediction vocabulary.
- It shapes how spelling, morphology, Unicode, and rare words are represented.

Character, syllable, word, subword

- Character: maximal coverage, but often long sequences.
- Syllable: linguistically meaningful unit for many Myanmar scripts and useful for preprocessing.
- Word: compact when segmentation is reliable, but word boundaries can be ambiguous.
- Subword: data-driven compromise between vocabulary size and sequence length.

BPE (Byte Pair Encoding)

- Start from small units.
- Merge frequent adjacent units.
- Store a deterministic vocabulary and merge rules.

SentencePiece

- Can train directly from raw sentences.
- Language-independent preprocessing assumptions.
- Supports BPE and unigram-style subword modeling.

Myanmar tokenization challenge: use more than words and characters

- **Word:** useful when reliable word boundaries exist; Myanmar whitespace does not always provide them.
- **Character:** script-internal, but a single Myanmar syllable may contain several Unicode characters.
- **Syllable:** a practical Myanmar-native unit; easy to segment with structural rules.
- **Subword:** learned BPE / unigram units; flexible, but tokenizer-dependent.
- **Byte:** universal fallback, but often inefficient for Myanmar.

syllbreak4all – RE-based syllable breaking for Burmese, Shan, Pa'o, Pwo Kayin, S'gaw Kayin, Rakhine, Myeik, Dawei, and Mon; reported 139,136 sentences / 1,910,822 syllables in the study corpus.

Myanmar syllable unit: what makes it useful?

မြန်မာစာ

- A Myanmar syllable is formed from a consonant plus optional medials, vowels, and dependent signs.
- Unicode stores many components in logical order even when rendering places marks visually elsewhere.
- Syllable segmentation gives a stable intermediate representation before word segmentation, MT, G2P, romanization, and other downstream tasks.
- It is also a useful tokenization candidate for LM experiments: compare syllable-level models against character and learned subword models.

sylbreak: one regular-expression boundary rule

Core idea from the sylbreak tool

```
$line = s/((?<!( $\$ssSymbol$ )[ $\$myConsonant$ ](?! [ $\$aThat\$ssSymbol$ ]))| [ $\$enChar\$otherChar$ ])/ $sep$1/g;
```

- Insert a boundary before a consonant when it is not part of a stacked consonant and is not followed by the aThat sign.
- The algorithm exploits Myanmar Unicode structure rather than learning a large statistical model.
- Implementations are available in Perl, Python, Java, JavaScript, and other forms through the project repositories.

Reference – <https://github.com/ye-kyaw-thu/sylbreak>

6. From pretrained LM to LLM

Scale, data, transfer, alignment

Pretraining changes the workflow

- Train once on broad data.
- Reuse the resulting representation for many tasks.
- Shift expensive learning from per-task training to a shared foundation model.

BERT vs GPT: two pretraining philosophies

- BERT: masked-token prediction for bidirectional representations.
- GPT: causal next-token prediction for direct generation.
- The distinction shaped the architecture and downstream use cases.

GPT-3 and in-context learning

- A sufficiently large causal LM can infer a task from examples in its prompt.
- No parameter update is needed for each new task.
- This made the prompt itself a lightweight control interface.

Scaling laws

- Loss often follows smooth power-law trends with model size, data, and compute.
- Scaling made capability more predictable.
- But scaling the wrong thing can waste compute.

Chinchilla-style compute optimality

- A fixed compute budget should balance model size and training tokens.
- Under-training a huge model can be less efficient than training a smaller model on more tokens.
- This reframed 'bigger model' toward 'better allocation of compute'.

7. Efficient adaptation and inference

How researchers get useful LMs without retraining everything

Full fine-tuning

- Update all parameters on a task or domain corpus.
- Simple conceptually, expensive for large models.
- Can cause catastrophic shifts when the adaptation set is narrow.

LoRA (Low-Rank Adaptation)

- Freeze the pretrained weights.
- Learn low-rank update matrices inside selected linear layers.
- Greatly reduces trainable parameters and optimizer memory.

QLoRA (Quantized Low-Rank Adapters)

- Quantize the frozen base model to low precision.
- Backpropagate through the quantized base into LoRA adapters.
- Makes much larger models feasible on modest hardware.

KV cache

- Autoregressive decoding repeatedly reuses prior keys and values.
- Caching avoids recomputing all attention states for each generated token.
- Memory grows with context length and batch size.

FlashAttention

- The exact attention math stays the same.
- The implementation is reorganized to reduce GPU memory traffic.
- The lesson: algorithmic efficiency includes hardware IO, not only big-O notation.

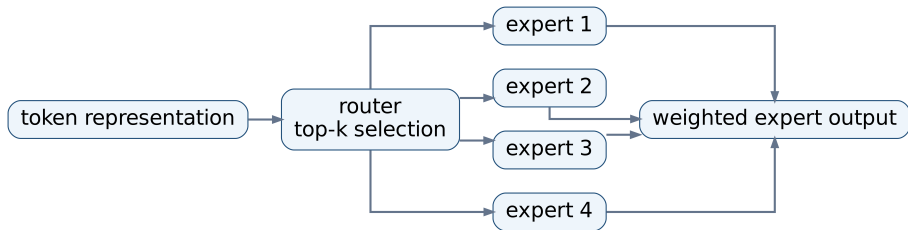
Speculative decoding

- A small draft model proposes several tokens.
- The large model verifies them in batches.
- Accepted tokens preserve the target model's distribution while reducing latency.

8. Mixture-of-Experts and alternative sequence models

Scale capacity without making every token use every parameter

Sparse Mixture-of-Experts



A router selects only a few experts for each token; total parameter count can be much larger than active compute.

Switch / Mixtral lesson

- Sparse routing can preserve compute budgets while increasing capacity.
- Routing introduces communication and load-balancing challenges.
- Open MoE families made this an important practical design point.

9. Modern reasoning and post-training

The model can improve by learning how to think, act, verify, and spend compute

Instruction tuning

- Convert generic continuation into helpful task following.
- Supervised demonstrations shape response format and behavior.
- Quality matters more than raw quantity for many post-training datasets.

DPO (Direct Preference Optimization)

- Optimize preference pairs directly without a separate online RL loop.
- Conceptually simpler than PPO-style RLHF (Reinforcement Learning from Human Feedback).
- PPO: Proximal Policy Optimization
- DPO converts preference pairs directly into a classification-like optimization objective.

Chain-of-thought prompting

- Ask for intermediate reasoning traces.
- Large models can gain substantial performance on multistep reasoning tasks.
- Prompting and training-time reasoning supervision are different mechanisms.

Test-time compute

- Spend more inference compute on hard problems.
- Search, sample, verify, or revise instead of decoding once.
- This creates a second scaling axis besides parameter count.

DeepSeek-R1: reasoning as a post-training objective

- Shows a 2025 research direction centered on large-scale RL for reasoning.
- R1-Zero demonstrates reasoning behavior can emerge under RL; R1 adds cold-start data and multi-stage training for readability and robustness.
- Open distilled variants connect reasoning research to smaller deployable models.

10. Retrieval-Augmented Generation

Add external, updateable knowledge rather than pretending the model is a database

Basic RAG pipeline



Documents are indexed separately; the query retrieves evidence that is inserted into the generator's context.

Dense retrieval

- Encode query and passages into vectors.
- Score similarity, often cosine or dot-product.
- Retrieve top-k candidates before generation.

Hybrid retrieval

- Sparse term matching catches exact names and rare strings.
- Dense retrieval captures semantic similarity.
- Combining them often improves recall and robustness.

Reranking

- Retrieve a wider candidate set cheaply.
- Use a stronger cross-encoder or late-interaction model to rerank.
- Send only the best evidence to the LLM.

Practical notebook: Passage RAG

- Sentence-Transformer embeddings.
- Cosine similarity for top-k retrieval.
- Retrieved Myanmar passages injected into a causal LM prompt.

RAG failure modes

- Retrieved the wrong evidence.
- Retrieved the right evidence but too late / too deep in the context.
- LLM ignored the evidence or hallucinated beyond it.
- The source itself is outdated or contradictory.

Long context is a resource, not a guarantee

- Larger windows can reduce retrieval engineering.
- But cost and latency rise.
- Models can still underuse information placed in the middle of long prompts.

GraphRAG

- Build an entity / relationship graph from documents.
- Create community summaries.
- Useful for corpus-level questions such as themes and relationships, where single-hop retrieval is weak.

RAG evaluation has at least two layers

- Retrieval: did we find useful evidence?
- Generation: did we answer faithfully using that evidence?
- End-to-end score alone hides where the system failed.

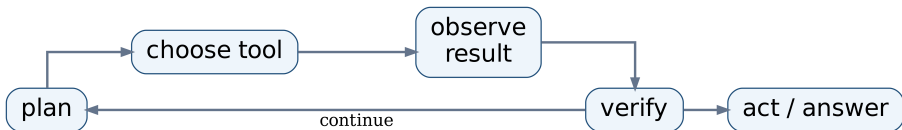
RAGAS / RAGChecker

- Use reference-free or diagnostic metrics where possible.
- Evaluate context relevance, faithfulness, and answer quality separately.
- Build error taxonomies, not only a single aggregate score.

12. Agents, tools, and the second brain

Move from 'answer generation' to stateful, tool-using workflows

Agent loop

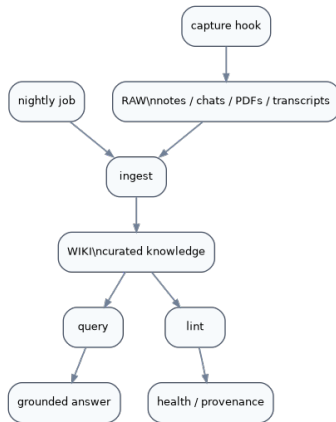


A useful agent loop plans, chooses tools, observes results, verifies progress, and either acts again or returns a final answer.

ReAct

- Interleave reasoning and action.
- The environment becomes a source of new information.
- This reduces the assumption that the model must know everything internally.

A practical 'second brain' pattern



The referenced Second Brain project separates immutable raw intake from curated wiki knowledge, with capture hooks, ingestion, grounded queries, linting, and scheduled maintenance.

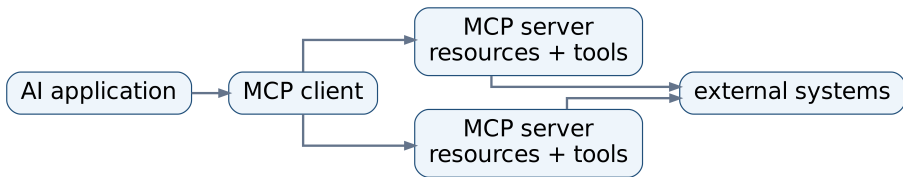
Grounded query as a contract

- Answer only from the curated knowledge layer.
- Cite source pages.
- Surface contradictions.
- Say when the knowledge base does not know.

13. MCP and composable tool ecosystems

A systems-level view of LLM applications

Model Context Protocol as an integration layer



MCP is an open standard introduced by Anthropic for connecting AI applications with data sources and tools through clients and servers.

Prefill vs decode

- Prefill processes the input context.
- Decode generates one token at a time.
- Their compute and memory bottlenecks differ.

Latency metrics

- Time to first token (TTFT).
- Inter-token latency / tokens per second.
- End-to-end latency.
- Throughput under concurrent load.

Prompt injection

- Retrieved documents and tool outputs can contain instructions.
- Treat untrusted text as data, not policy.
- Keep tool permissions and system instructions outside retrieved content.

Practical 1: environment check

Notebook / script pattern

```
torch.cuda.is_available() print(torch.cuda.get_device_name(0))
```

Engineering reality: hardware, memory, software versions, and reproducibility constrain LM experiments.

Practical 3: KenLM

Notebook / script pattern

```
lmplz -> ARPA -> build_binary -> perplexity
```

Pipeline: counts -> smoothing -> model format -> scoring.

Practical 5: LSTM

Notebook / script pattern

Embedding -> LSTM -> Linear -> CrossEntropy

Pipeline: embedding -> LSTM -> linear output -> cross-entropy.

Practical 6: XGLM

Notebook / script pattern

```
AutoTokenizer + AutoModelForCausalLM Trainer(...)
```

Pipeline: tokenizer -> pretrained causal LM -> fine-tuning -> generation.

Practical 8: generation

Notebook / script pattern

```
temperature=0.7 top_k=50 top_p=0.95
```

Decoding study: temperature, top-k, and top-p change the sampled distribution.

Practical 9: Passage RAG

Notebook / script pattern

```
encode(query) -> cosine -> top-k passages -> prompt -> LLM
```

Diagnostic: separate retrieval quality from generation quality.

Integrated Myanmar LM experiment

Core LM comparison

- KenLM 5-gram vs PyTorch LSTM vs XGLM.
- Explain the observed PPL gap through tokenization and data regime.
- Compare intrinsic metrics with qualitative generations.

Myanmar creative case: myPoetry

- 393 poets; 1,873 poem titles; 46,933 poetry lines.
- Publicly released corpus with SRILM 5-gram and GPT-2 generations.
- Poetry data changes the target distribution; evaluate fluency, repetition, style, and domain fit.

Reference – <https://github.com/ye-kyaw-thu/myPoetry>

16. Myanmar-first research agenda

Turn the course into a research launchpad

Research ideas for master's theses

- Syllable vs BPE/unigram tokenizer benchmark for Myanmar NLP and MT.
- Domain-adaptive LM interpolation vs LoRA.
- **myPoetry**: statistical LM vs GPT-style generation with poetry-aware evaluation.
- RAG chunking / retrieval benchmark for Myanmar documents.
- Evidence-grounded agent or second-brain assistant for lab workflows.

17. Final synthesis

One language-model equation, many engineering choices

References 1

- **1. Shannon (1948)** A Mathematical Theory of Communication.
<https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>
- **2. Katz (1987)** Estimation of Probabilities from Sparse Data for the Language Model Component of a Speech Recognizer.
<https://aclanthology.org/P87-1015/>
- **3. Kneser & Ney (1995)** Improved backing-off for m-gram language modeling.
https://www-i6.informatik.rwth-aachen.de/web/Maus/Kneser1995_ImprovedBackingOff.pdf
- **4. Bengio et al. (2003)** A Neural Probabilistic Language Model.
<https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf>
- **5. Hochreiter & Schmidhuber (1997)** Long Short-Term Memory.
<https://www.bioinf.jku.at/publications/older/2604.pdf>
- **6. Mikolov et al. (2010)** Recurrent Neural Network Based Language Model.
<https://www.fit.vut.cz/research/publication/9473>
- **7. Sutskever et al. (2014)** Sequence to Sequence Learning with Neural Networks.
<https://arxiv.org/abs/1409.3215>
- **8. Bahdanau et al. (2015)** Neural Machine Translation by Jointly Learning to Align and Translate.
<https://arxiv.org/abs/1409.0473>

References 2

- **9. Vaswani et al. (2017)** Attention Is All You Need.
<https://arxiv.org/abs/1706.03762>
- **10. Devlin et al. (2019)** BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
<https://arxiv.org/abs/1810.04805>
- **11. Radford et al. (2019)** Language Models are Unsupervised Multitask Learners (GPT-2 report).
https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- **12. Raffel et al. (2020)** Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.
<https://arxiv.org/abs/1910.10683>
- **13. Brown et al. (2020)** Language Models are Few-Shot Learners.
<https://arxiv.org/abs/2005.14165>
- **14. Kaplan et al. (2020)** Scaling Laws for Neural Language Models.
<https://arxiv.org/abs/2001.08361>
- **15. Hoffmann et al. (2022)** Training Compute-Optimal Large Language Models (Chinchilla).
<https://arxiv.org/abs/2203.15556>
- **16. Ouyang et al. (2022)** Training language models to follow instructions with human feedback.
<https://arxiv.org/abs/2203.02155>

References 3

- **17. Lewis et al. (2020)** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
<https://arxiv.org/abs/2005.11401>
- **18. Reimers & Gurevych (2019)** Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.
<https://arxiv.org/abs/1908.10084>
- **19. Kudo & Richardson (2018)** SentencePiece: A simple and language independent subword tokenizer.
<https://aclanthology.org/D18-2012/>
- **20. Sennrich et al. (2016)** Neural Machine Translation of Rare Words with Subword Units.
<https://arxiv.org/abs/1508.07909>
- **21. Hu et al. (2021)** LoRA: Low-Rank Adaptation of Large Language Models.
<https://arxiv.org/abs/2106.09685>
- **22. Dettmers et al. (2023)** QLoRA: Efficient Finetuning of Quantized LLMs.
<https://arxiv.org/abs/2305.14314>
- **23. Dao et al. (2022)** FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.
<https://arxiv.org/abs/2205.14135>
- **24. Kwon et al. (2023)** Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM).
<https://arxiv.org/abs/2309.06180>

References 4

- **25. Leviathan et al. (2023)** Fast Inference from Transformers via Speculative Decoding.
<https://arxiv.org/abs/2211.17192>
- **26. Fedus et al. (2022)** Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.
<https://arxiv.org/abs/2101.03961>
- **27. Jiang et al. (2024)** Mixtral of Experts.
<https://arxiv.org/abs/2401.04088>
- **28. Gu & Dao (2024)** Mamba-2 / Structured State Space Duality.
<https://proceedings.mlr.press/v235/dao24a.html>
- **29. Wei et al. (2022)** Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.
<https://arxiv.org/abs/2201.11903>
- **30. Wang et al. (2023)** Self-Consistency Improves Chain of Thought Reasoning in Language Models.
<https://arxiv.org/abs/2203.11171>
- **31. Rafailov et al. (2023)** Direct Preference Optimization: Your Language Model is Secretly a Reward Model.
<https://arxiv.org/abs/2305.18290>
- **32. Wang et al. (2023)** Self-Instruct: Aligning Language Models with Self-Generated Instructions.
<https://arxiv.org/abs/2212.10560>

References 5

- **33. Yao et al. (2023)** ReAct: Synergizing Reasoning and Acting in Language Models.
<https://arxiv.org/abs/2210.03629>
- **34. Schick et al. (2023)** Toolformer: Language Models Can Teach Themselves to Use Tools.
<https://arxiv.org/abs/2302.04761>
- **35. Park et al. (2023)** Generative Agents: Interactive Simulacra of Human Behavior.
<https://arxiv.org/abs/2304.03442>
- **36. Liu et al. (2024 version)** Lost in the Middle: How Language Models Use Long Contexts.
<https://arxiv.org/abs/2307.03172>
- **37. Edge et al. (2024)** From Local to Global: A Graph RAG Approach to Query-Focused Summarization.
<https://arxiv.org/abs/2404.16130>
- **38. Asai et al. (2024)** Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.
<https://arxiv.org/abs/2310.11511>
- **39. Es et al. (2023)** RAGAS: Automated Evaluation of Retrieval Augmented Generation.
<https://arxiv.org/abs/2309.15217>
- **40. Ru et al. (2024)** RAGChecker: A Fine-grained Framework for Diagnosing RAG.
<https://arxiv.org/abs/2408.08067>

References 6

- **41. Santhanam et al. (2022)** ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction.
<https://aclanthology.org/2022.naacl-main.272/>
- **42. Gao et al. (2023)** Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE).
<https://arxiv.org/abs/2212.10496>
- **43. Liu et al. (2023)** AgentBench: Evaluating LLMs as Agents.
<https://arxiv.org/abs/2308.03688>
- **44. Touvron et al. (2023)** LLaMA: Open and Efficient Foundation Language Models.
<https://arxiv.org/abs/2302.13971>
- **45. Yang et al. (2025)** Qwen3 Technical Report.
<https://arxiv.org/abs/2505.09388>
- **46. DeepSeek-AI et al. (2025)** DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.
<https://arxiv.org/abs/2501.12948>
- **47. Gemma Team (2025)** Gemma 3 Technical Report.
<https://arxiv.org/abs/2503.19786>
- **48. Anthropic (2024)** Introducing the Model Context Protocol.
<https://www.anthropic.com/news/model-context-protocol>

References 7

- **49. Second Brain project (2026)** Obsidian + Claude Code + Karpathy LLM Wiki pattern.
<https://aquinas-protocol.github.io/second-brain/>
- **50. Yao et al. (2026)** VecInfer: Efficient LLM Inference with Low-Bit KV Cache.
<https://aclanthology.org/2026.acl-long.1454/>
- **51. Yang et al. (2026)** Faster MoE LLM Inference for Extremely Large Models.
<https://aclanthology.org/2026.findings-acl.2140/>
- **52. Yang et al. (2026)** LongSpec: Long-Context Lossless Speculative Decoding.
<https://aclanthology.org/2026.acl-long.83/>
- **53. Liao & Wen (2026)** Channel-Aware Mixed-Precision Quantization for Efficient Long-Context Inference.
https://proceedings.iclr.cc/paper_files/paper/2026/hash/55f5aa8d997a8071d0651e0009ec49a2-Abstract-Conference.html

References 8

- **54. Ye Kyaw Thu (GitHub);** sylbreak: Syllable segmentation tool for Myanmar (Burmese). <https://github.com/ye-kyaw-thu/sylbreak>
- **55. Ye Kyaw Thu et al. (2021);** sylbreak4all: Regular Expressions for Syllable Breaking of Nine Major Ethnic Languages of Myanmar. iSAI-NLP 2021, pp. 1–6, DOI: 10.1109/iSAI-NLP54397.2021.9678188. <https://github.com/ye-kyaw-thu/sylbreak4all>
- **56. Ye Kyaw Thu et al. (2013);** A Study of Myanmar Word Segmentation Schemes for Statistical Machine Translation. ICCA 2013, pp. 167–179. <https://github.com/ye-kyaw-thu/sylbreak>
- **57. Ye Kyaw Thu et al. (2016);** A Large-scale Study of Statistical Machine Translation Methods for Myanmar Language. SNLP 2016. <https://github.com/ye-kyaw-thu/sylbreak>
- **58. Ye Kyaw Thu et al. (2023);** myPoetry: Myanmar Poetry Corpus. GitHub repository, Version 1.0. <https://github.com/ye-kyaw-thu/myPoetry>